



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Impact on programming

3.4 Scalability

Sergi Abadal (abadal@ac.upc.edu)

Xarxes i Jocs On-line – 2025-2026
Grau en Disseny i Desenvolupament de Videojocs

1. Introduction
2. The protocol stack
3. Impact on programming of on-line games (4 sessions)
 - Summary and general aspects
 - Latency, jitter, and reliability
 - Latency hiding techniques
 - **Scalability**
4. Other stories

Remember from last class



- A dumb terminal is the easiest way to replicate state in a client-server structure. Clients just update when the server tells them to (with $RTT/2$ delay).
- We can hide the effects of latency by performing some actions at the client and server
 - Client side interpolation: from old state ($i-2$) to last state by the server ($i-1$) we apply interpolation. Adds lag, but safe.
 - Client side prediction: from last state by the server ($i-1$) to present we extrapolate. Reduces lag, but leads to errors.
 - Server side rewind: servers need to re-calculate actions with recent inputs from clients, which are not instantaneous



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Scalability

Halo has to solve the hard problem

- Highly competitive multiplayer action game
- 16 players, vehicles, hundreds of replicated objects
- No dedicated servers
- Game is expected to work regardless of connection quality
- For N players, $O(N^2)$ data needs to be networked

The techniques we have seen so far allow us to

- Replicate the world state in a relatively smart way (object replication: world state deltas and object deltas)
- Program our own streamlined methods to order, acknowledge and prioritize packets (custom reliability)
- Reduce the amount of data needed to transmit each object (object or snapshot compression)
- Reduce the number of states sent per second while still having a satisfying experience (interpolation, prediction)

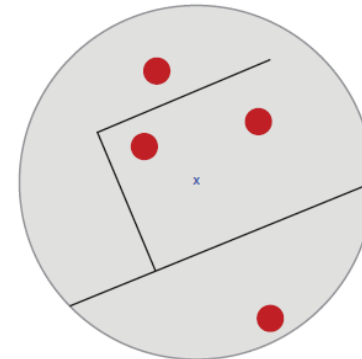
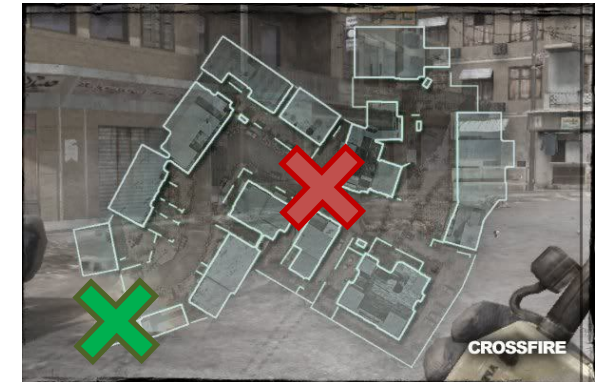
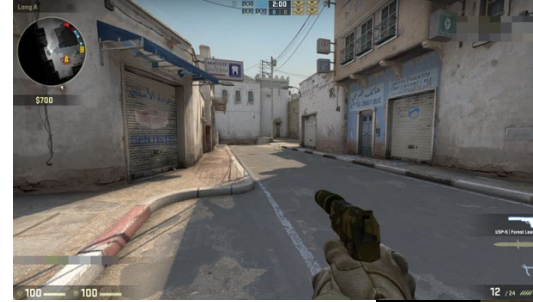
- These may enable the scaling of our game.
- Here, scaling means making the game equally playable, with a very similar quality of experience, regardless of the number of simultaneous players.
- In this class, we discuss different techniques:
 - Object scope and relevancy
 - Prioritization and frequency
 - Static zones and server partitioning
 - Instancing

- These may enable the scaling of our game.
- Here, scaling means making the game equally playable, with a very similar quality of experience, regardless of the number of simultaneous players.
- In this class, we discuss different techniques:
 - **Object scope and relevancy**
 - Prioritization and frequency
 - Static zones and server partitioning
 - Instancing

- An object is considered in scope or relevant for a particular client when that client should be informed about updates to the object in question
 - Scope is generally assessed by proximity: close objects to a client are in its scope. Far away objects will probably not.
 - Relevancy may be more difficult to assess: health of player is relevant always, and relevance of other objects may vary.
- By not updating the objects that are not in scope or updating less frequently those that are less relevant, we save precious resources.

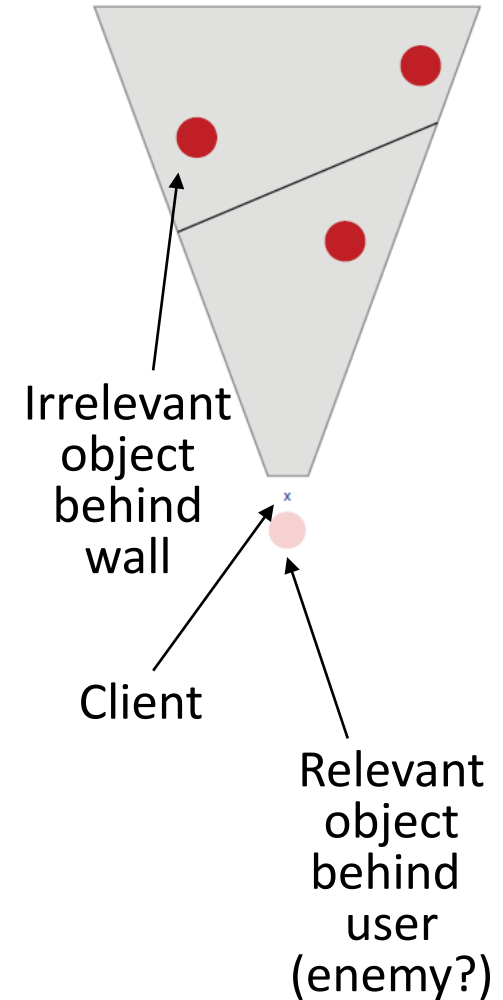
Object scope and relevancy

- Object scope is generally assessed using a spatial approach, but distance only is not enough to determine relevancy.
 - May depend on the weapon
 - May depend on the position in map
 - May depend on the player view and surroundings (walls)
- You can use techniques similar **visibility culling**



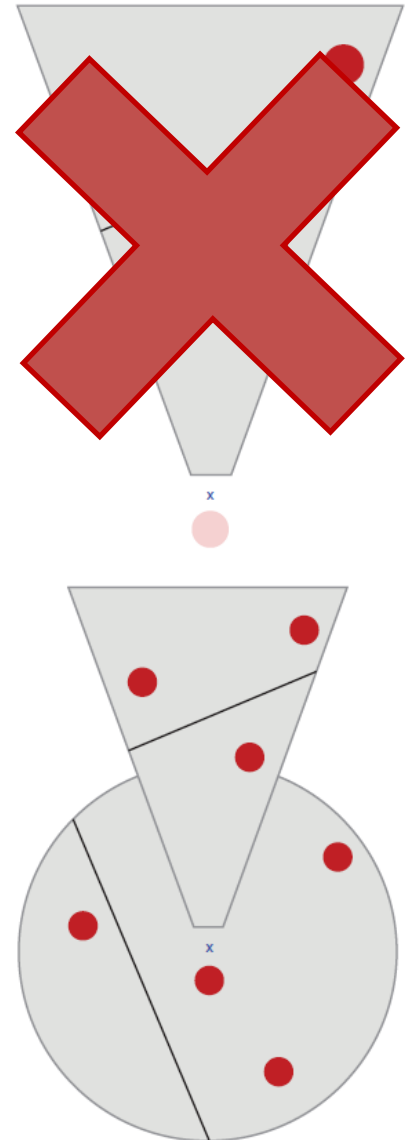
Using the View Frustum

- The view frustum is typically used in visibility culling: if outside the frustum, not visible and therefore not treated.
- We can do the same with the replicated objects.
- However, doing only this will create problems:
 - Turning around suddenly adds delay: sends the new orientation to server, server sends objects back
 - You want objects near and behind you to be accessible as soon as possible, especially enemies
 - Objects behind walls are still not relevant...



Using the View Fustrum

- The view frustum is typically used in visibility culling: if outside the frustum, not visible and therefore not treated.
- We can do the same with the replicated objects.
- This is generally done combined with distance
 - Object in frustum are considered
 - Objects nearby in 360° turns are considered
 - Quick turns would still work with nearby objects

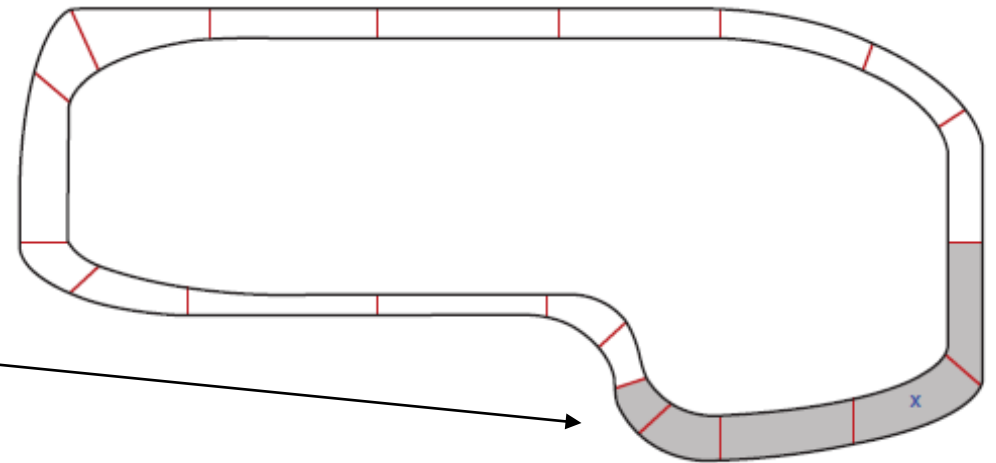


Other visibility techniques

- We can also use the **Potentially Visible Set (PVS)**: non-frustum non-static areas that we expect to be able to see.

Examples in a racing game

- Flat straight: a lot of visibility
- Uphill straight: lower visibility
- Turning road: low visibility



- This can be used in corridor-based shooters, where doors become **portals** extending the PVS beyond the room.

- There are cases where non-visible objects are still relevant and should be accounted for. We can define different scope/relevancy profiles for different types of objects.
- Example: grenades
 - You want to hear explosions that happen in nearby rooms, especially if you have thrown it.
 - Thus, grenades should be replicated if they are in nearby rooms.
 - Replication can occur based on radius rather than on visibility.
 - Replication can be complete or partial (just sound or effects).

- These may enable the scaling of our game.
- Here, scaling means making the game equally playable, with a very similar quality of experience, regardless of the number of simultaneous players.
- In this class, we discuss different techniques:
 - Object scope and relevancy
 - **Prioritization and frequency**
 - Static zones and server partitioning
 - Instancing

- Some times, the users are the bottleneck as they cannot receive too many objects per second (bandwidth)
 - Mobile games with varying connection conditions
 - Games with huge number of objects
- Besides only sending object deltas, we can also use **prioritization and multiple update frequencies**
 - Objects with higher priority (relevance) are replicated first until the bandwidth cap is met. If there is space for lower priority objects, then we also replicate them.
 - Equally, we can assign different frequencies. Low priority objects will be replicated less often.

More details at:

<https://youtu.be/h47zZrqjgLC?t=1195>

Replication is never fully reliable

- Unreliability enables aggressive *prioritization*, which lets us handle the richness of our simulation
- Flow control layer decides when to send a packet, and what size it should be
- Replication writes data into the packet until full
- There is always more data than will fit, so we write high-priority data first

More details at:

<https://youtu.be/h47zZrqjgLC?t=1195>

Prioritization

- Priority is based on client view and simulation state
- Priority is calculated separately per-object per-client
- Distance/direction is the core metric
- Size & speed affect priority
- Shooting & damage apply appropriate boosts
- Lots of special cases (e.g. thrown grenades)

The HALO use case

More details at:

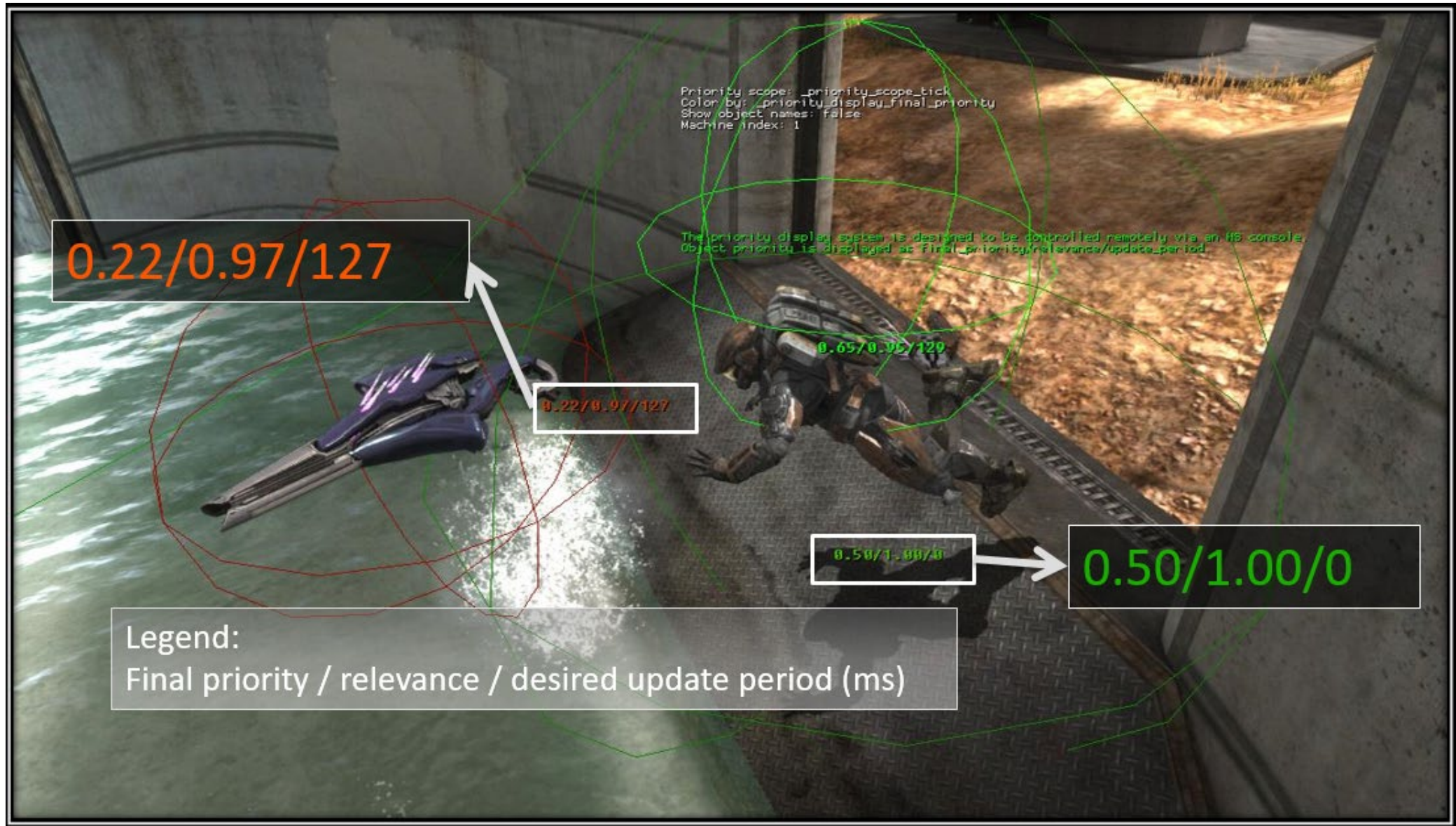
<https://youtu.be/h47zZrqjgLC?t=1195>



The HALO use case

More details at:

<https://youtu.be/h47zZrqjgLC?t=1195>



The HALO use case



CITM

More details at:

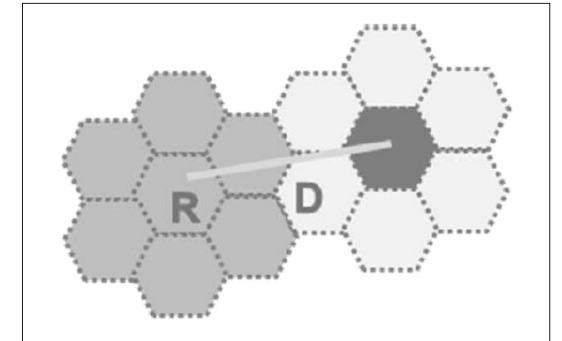
<https://youtu.be/h47zZrqjgC?t=1195>



- These may enable the scaling of our game.
- Here, scaling means making the game equally playable, with a very similar quality of experience, regardless of the number of simultaneous players.
- In this class, we discuss different techniques:
 - Object scope and relevancy
 - Prioritization and frequency
 - **Static zones and server partitioning**
 - Instancing

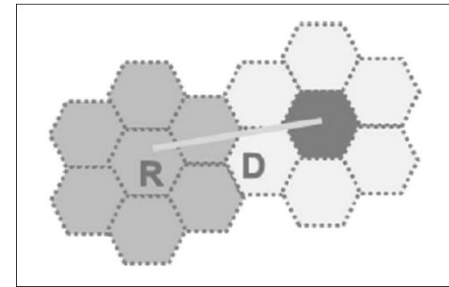
Static zones

- It is difficult to track very large scenarios with many players
- A way to improve scalability and facilitate tracking is to break down the map into static zones: a forest, a dungeon, a town
- Only objects in the same zone are replicated (if they are relevant)
- When changing zones, the server needs to make a transition (replicate all the newly relevant objects).



Static zones (II)

- Static zones are used in MMORPGs and not in action games. Besides the map size, the main reason are transitions. They can be:
 - Gradual: we somehow fade out/in objects at the boundary of the two areas, with delay.
 - Abrupt: we just show “loading new area”, which can still be acceptable in MMORPGS.
- Advantage is that different servers can be used for different zones.
- Disadvantage is that load may not be balanced (bad experience in crowded zones)



Server partitioning (or sharding)



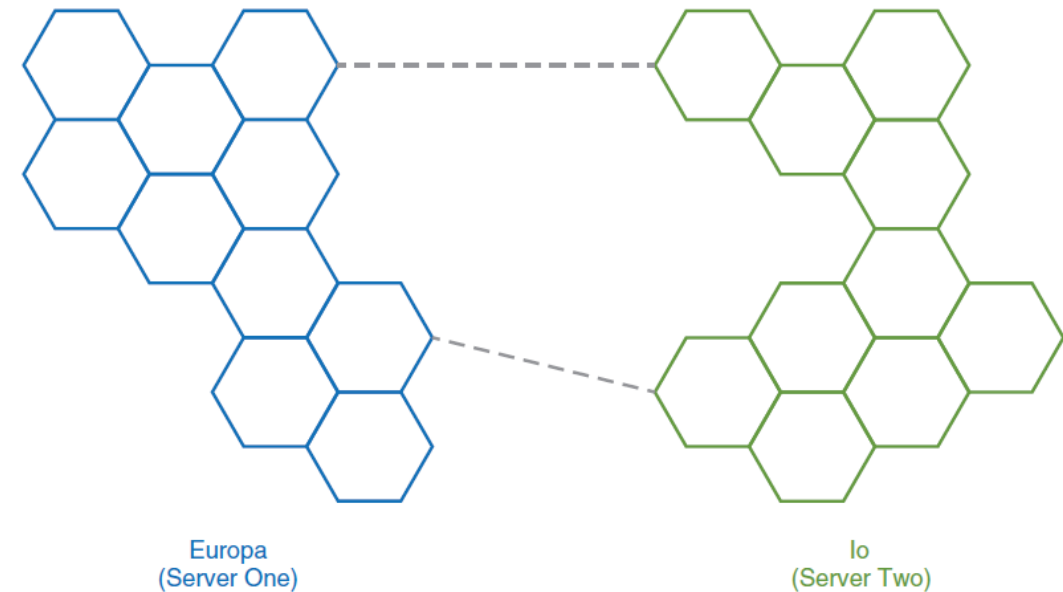
CITM

- It is quite widespread that big companies have data centers with many servers to keep up with demand
- With action games, each server is allocated one or several separate matches that are run in parallel.
- Allocation is based on a process of load balancing, to avoid servers not being to keep up.
- Results and other outcomes are later updated in a shared database.



Server partitioning (or sharding)

- This approach can be leveraged in massive games.
- We take several static zones and form a continent. The continent will run in a single server.
- Clients transitioning between continents will have a similar experience than with static zones (wait until client migrates and loads new objects)
- This works well if the load is balanced. However, this does not depend on the designer, but the player density.



- These may enable the scaling of our game.
- Here, scaling means making the game equally playable, with a very similar quality of experience, regardless of the number of simultaneous players.
- In this class, we discuss different techniques:
 - Object scope and relevancy
 - Prioritization and frequency
 - Static zones and server partitioning
 - **Instancing**

- Even considering an MMORPGs in a shared world, the idea is to create multiple instances of a zone where a number of users will play “privately” for a while
- This applies to dungeons, where separate raids of users may enter and run through it concurrently but not interfering with each other.
- Some games (*Star Wars: Old Republic*) also use instancing transparently in overcrowded areas of a single world, even if that means that two users in separate instances will not meet as intended.





UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Impact on programming

3.4 Scalability

Sergi Abadal (abadal@ac.upc.edu)

Xarxes i Jocs On-line – 2025-2026
Grau en Disseny i Desenvolupament de Videojocs